

LOG ANALYZER – MULTITHREADED LOG FILE PARSER

By- Dhairya Agrawal

reg no- 25BAI10758

1. Project Title

Log Analyzer – Multithreaded Log File Parser

2. Aim

To develop a Java-based multithreaded application that automatically parses, validates, and analyzes server log files and generates meaningful statistical information and a detailed analysis report.

3. Objective

The main objectives of the project are:

- To automate the analysis of server log files.
 - To process multiple log entries efficiently using multithreading.
 - To validate log entries using Regular Expressions.
 - To identify and count corrupted or invalid log entries.
 - To calculate the average response time.
 - To identify the slowest request.
 - To detect requests taking more than 500 milliseconds.
 - To analyze requests based on log level.
 - To identify the top five IP addresses.
 - To identify the top five frequently accessed endpoints.
 - To generate an automated analysis report.
 - To demonstrate thread-safe programming in Java.
-

4. Problem Statement

Server applications continuously generate log files containing information about user requests, IP addresses, endpoints, response times, and system activities.

When these files become large, manually examining them becomes time-consuming and inefficient. There is a need for an automated system that can process large numbers of log entries, detect invalid records, and provide useful statistical information.

The proposed Log Analyzer solves this problem by automatically reading the log file, dividing the workload among multiple threads, validating each record, analyzing the valid entries, and generating a structured report.

5. Concept

The project is based on the concept of **multithreaded log processing**.

The complete log file is divided into multiple chunks. Four worker threads process these chunks concurrently. Each worker validates and parses its assigned log entries.

A shared statistics object stores the results of the analysis. Since multiple threads access the same statistics, thread-safe Java classes such as ConcurrentHashMap, AtomicInteger, AtomicLong, and synchronized collections are used.

The application also uses a Regular Expression to identify whether a log entry follows the expected format.

The main concepts demonstrated are:

- File Handling
- Multithreading
- Thread Pool
- Runnable Interface
- Regular Expressions
- Exception Handling
- Custom Exceptions
- Concurrent Collections
- Atomic Variables
- Data Processing
- Report Generation

6. Requirements

6.1 Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Minimum 500 MB available storage
- Standard keyboard and display

6.2 Software Requirements

- Java Development Kit (JDK)
- Java compiler
- Command Prompt / Terminal
- Any Java-supported operating system
- Text editor or Java IDE

6.3 Input Requirements

The application requires a server log file containing entries in the expected format.

Example:

```
2026-09-18 10:15:23 [INFO] 192.168.1.10 /api/login "User login successful"  
resp_time_ms=120
```

7. Functional Requirements

The system should be able to:

1. Read a server log file.
2. Check whether the input file exists.
3. Read all log entries.
4. Divide the entries into processing chunks.
5. Create multiple worker threads.
6. Validate log entries.
7. Parse valid entries.
8. Detect corrupted entries.
9. Calculate response-time statistics.

10. Count requests by log level.
 11. Count requests by IP address.
 12. Count requests by endpoint.
 13. Identify slow requests.
 14. Display analysis results.
 15. Generate a detailed report.
-

8. Non-Functional Requirements

Performance

The application uses multiple threads to process different portions of the log file concurrently.

Reliability

Invalid log entries are handled without stopping the complete analysis process.

Maintainability

The project is divided into logical classes such as LogAnalyzer, LogParser, LogData, and Stats.

Thread Safety

Shared statistics are maintained using thread-safe data structures and atomic variables.

Usability

The application can be executed using simple command-line commands.

9. System Workflow

The system follows the following workflow:

Start



Read Command-Line Arguments



Check Input File



Read Log File



Check Whether File Is Empty



Create Statistics Object



Divide Log File into Chunks



Create 4 Worker Threads



Process Log Entries Concurrently



Validate Log Format



Parse Valid Entries



Count Invalid Entries



Update Thread-Safe Statistics



Wait for All Threads



Calculate and Display Results



Generate Analysis Report



End

10. Algorithm

Step 1

Start the program.

Step 2

Read the input filename and report filename from command-line arguments.

Step 3

If no filenames are provided, use server.log as the input file and analysis_report.txt as the output report.

Step 4

Check whether the input file exists.

Step 5

Read all lines from the input log file.

Step 6

If the file is empty, terminate the program.

Step 7

Create a shared Stats object.

Step 8

Set the number of worker threads to four.

Step 9

Divide the log entries into four processing chunks.

Step 10

Create a fixed thread pool using ExecutorService.

Step 11

Assign each chunk to a LogParser worker.

Step 12

For every log line, check whether it matches the required Regular Expression.

Step 13

If the log entry is valid, extract its timestamp, level, IP address, endpoint, message, and response time.

Step 14

Update the shared statistics.

Step 15

If the log entry is invalid, increment the corrupted-entry counter.

Step 16

Identify requests whose response time is greater than 500 milliseconds.

Step 17

Wait for all worker threads to complete.

Step 18

Display the analysis results.

Step 19

Generate the detailed analysis report.

Step 20

End the program.

11. Log Entry Format

The application expects the following format:

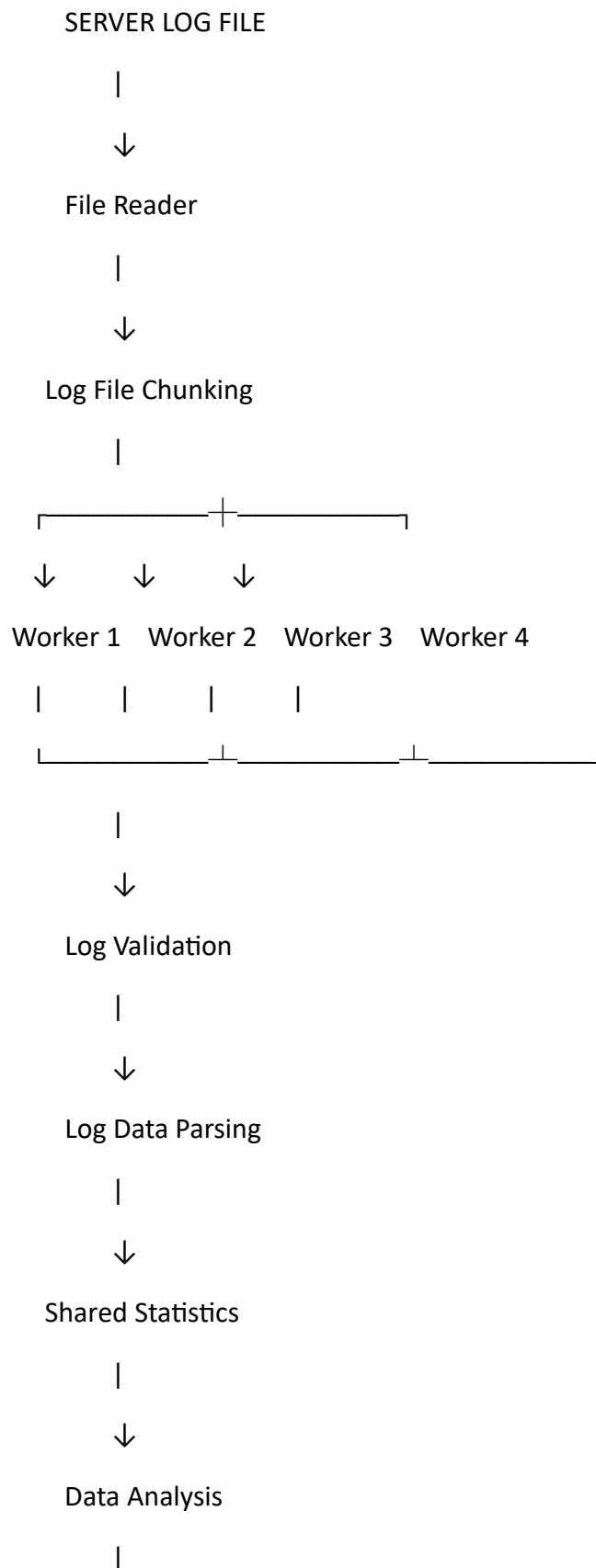
YYYY-MM-DD HH:MM:SS [LEVEL] IP_ADDRESS ENDPOINT "MESSAGE" resp_time_ms=TIME

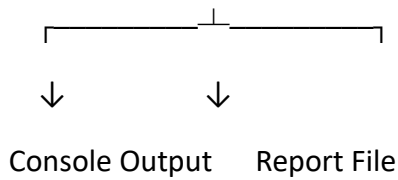
Components

Component	Description
Timestamp	Date and time of the request
Level	Log level such as INFO, DEBUG, ERROR, etc.
IP Address	Address of the requesting client
Endpoint	Requested server endpoint
Message	Description of the logged event
Response Time	Server response time in milliseconds

12. System Architecture

The system consists of the following major components:





13. Class Description

13.1 LogAnalyzer

This is the main class of the application.

Responsibilities:

- Accept command-line arguments.
- Read the input file.
- Create worker threads.
- Wait for processing to complete.
- Display analysis results.
- Generate the final report.

13.2 LogData

LogData stores the information of an individual valid log entry.

It contains:

- Timestamp
- Log level
- IP address
- Endpoint
- Message
- Response time

The class represents an immutable log record.

13.3 Stats

The Stats class stores and manages the analysis results.

It maintains:

- Total valid entries
- Corrupted entries
- Total response time
- Slowest response time
- Slow requests
- Requests by level
- Requests by IP address
- Requests by endpoint

Thread-safe data structures are used because multiple worker threads update the same object.

13.4 LogParser

LogParser implements the Runnable interface.

Each object represents a worker responsible for processing a specific chunk of log lines.

Its responsibilities include:

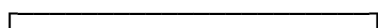
- Reading assigned lines
- Validating log format
- Parsing log entries
- Handling parsing exceptions
- Updating shared statistics

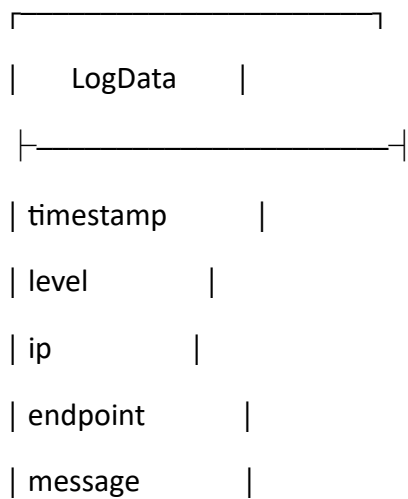
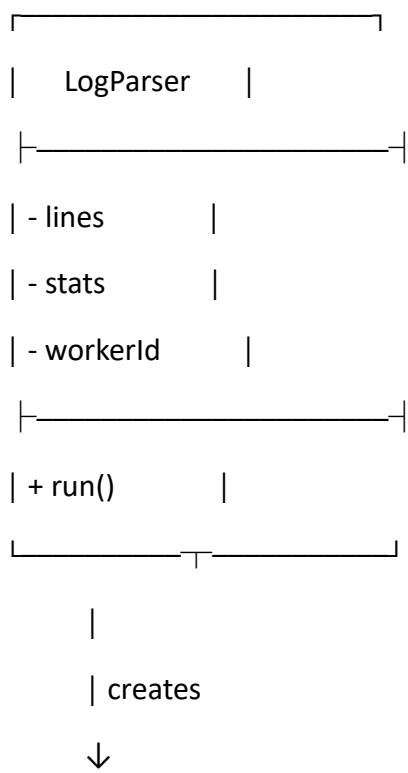
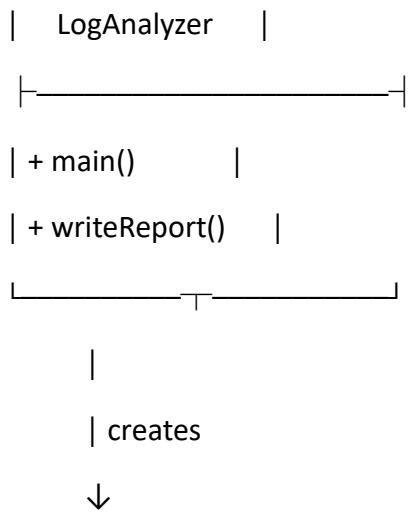
13.5 LogFormatException

This is a custom exception used to represent invalid log-format conditions.

It allows the program to distinguish expected log-format errors from other exceptions.

14. Class Diagram





```
| responseTime      |  
└──────────────────┘
```

```
┌──────────────────┐  
|   Stats   |  
└──────────┘
```

```
| byLevel      |  
| byIp        |  
| byEndpoint   |  
| slow        |  
| total       |  
| corrupt     |  
| totalTime   |  
| slowest     |  
└──────────┘
```

```
| + add()      |  
| + addCorrupt() |  
| + getTotal() |  
| + getCorrupt() |  
| + getSlowest() |  
└──────────┘
```

```
┌──────────────────┐  
| LogFormatException |  
└──────────┘  
| extends Exception |  
└──────────┘
```

15. Multithreading Implementation

The project uses a fixed thread pool containing four worker threads.

`ExecutorService pool = Executors.newFixedThreadPool(workerCount);`

The log file is divided into chunks, and each chunk is assigned to a worker.

The worker is implemented using the `Runnable` interface.

This allows multiple sections of the log file to be processed concurrently.

The application waits for the worker threads to finish before displaying the final analysis.

16. Thread-Safe Processing

Multiple worker threads update the same statistics object. Therefore, synchronization and atomic operations are required.

The project uses:

`ConcurrentHashMap`

`AtomicInteger`

`AtomicLong`

`Collections.synchronizedList()`

`AtomicInteger` and `AtomicLong` are used for counters and response-time calculations.

The slowest response time is updated using a Compare-And-Set operation to ensure safe concurrent modification.

17. Exception Handling

The application implements multiple levels of exception handling.

Custom Exception

`LogFormatException` is used when a log entry does not follow the required format.

Number Format Exception

If the response time cannot be converted into an integer, the entry is considered invalid.

IOException

File-reading and report-writing errors are handled using `IOException`.

InterruptedException

If thread processing is interrupted, the worker pool is shut down safely and the thread interruption status is restored.

18. Analysis Performed

The application performs the following analysis.

18.1 Valid Entries

Counts all correctly formatted log entries.

18.2 Invalid Entries

Counts entries that do not match the expected format.

18.3 Average Response Time

The average response time is calculated as:

Average Response Time =

Total Response Time / Total Valid Entries

18.4 Slowest Request

The application tracks the highest response time found among valid requests.

18.5 Slow Requests

Any request with:

Response Time > 500 ms

is classified as a slow request.

18.6 Requests by Log Level

The system counts the number of requests for each log level.

18.7 Top IP Addresses

The five IP addresses with the highest number of requests are identified.

18.8 Top Endpoints

The five most frequently accessed endpoints are identified.

19. Source Code

The complete source code is contained in:

LogAnalyzer.java

The main implementation includes:

- File input handling
 - Log parsing
 - Regular Expression validation
 - Custom exception handling
 - Multithreaded processing
 - Thread-safe statistics
 - Console output
 - Report generation
-

20. Sample Input

A sample server.log file can contain entries such as:

```
2026-09-18 10:15:23 [INFO] 192.168.1.10 /api/login "User login successful"
resp_time_ms=120
```

```
2026-09-18 10:16:11 [DEBUG] 192.168.1.15 /api/users "Fetching user information"
resp_time_ms=240
```

```
2026-09-18 10:17:05 [ERROR] 192.168.1.20 /api/payment "Payment processing failed"
resp_time_ms=850
```

```
2026-09-18 10:18:42 [INFO] 192.168.1.10 /api/products "Product list requested"
resp_time_ms=180
```

```
2026-09-18 10:19:30 [WARNING] 192.168.1.25 /api/orders "High response time detected"
resp_time_ms=620
```

An invalid entry may also be present to test error handling.

21. Execution

Compile

```
javac LogAnalyzer.java
```

Run with Default Files

```
java LogAnalyzer
```

The default input file is:

```
server.log
```

The default output file is:

```
analysis_report.txt
```

Run with Custom Files

```
java LogAnalyzer server.log report.txt
```

22. Expected Output

The program displays information in the following format:

```
=== Log Analyzer Started ===
```

```
Input file: server.log
```

```
Report file: analysis_report.txt
```

```
Lines read: <number>
```

```
Worker-1 finished: <number> lines
```

```
Worker-2 finished: <number> lines
```

```
Worker-3 finished: <number> lines
```

```
Worker-4 finished: <number> lines
```

```
***** Analysis Results *****
```

```
Valid entries: <number>
```

```
Invalid entries: <number>
```

```
Average response time: <value> ms
```

```
Slowest request: <value> ms
```

Slow requests (>500ms): <number>

--- Requests by Level ---

--- Top 5 IP Addresses ---

--- Top 5 Endpoints ---

Report saved to: analysis_report.txt

The actual values depend on the contents of the input log file.

23. Output Report

The application generates:

analysis_report.txt

The report contains:

- Source file
- Number of worker threads
- Valid entries
- Invalid entries
- Average response time
- Slowest request
- Number of slow requests
- Requests by log level
- Top five IP addresses
- Top five endpoints
- Slow request details

24. Testing

The application can be tested using different input conditions.

Test Case 1 – Valid Log File

Input: A properly formatted server log file.

Expected Result: All valid entries are parsed and included in the analysis.

Test Case 2 – Invalid Log Entry

Input: A log entry that does not match the expected format.

Expected Result: The entry is counted as a corrupted entry.

Test Case 3 – Empty File

Input: An empty server log file.

Expected Result: The program displays a warning and terminates without performing analysis.

Test Case 4 – Missing File

Input: A filename that does not exist.

Expected Result: The program displays an appropriate file-not-found message.

Test Case 5 – Slow Request

Input: A valid request with response time greater than 500 ms.

Expected Result: The request is included in the slow-request analysis.

25. Project Structure

LogAnalyzer/

|

├─ LogAnalyzer.java

├─ server.log

├─ analysis_report.txt

├─ README.md

└─ statement.md

26. Advantages

- Reduces manual log analysis.
 - Processes multiple log entries concurrently.
 - Detects invalid log entries.
 - Provides useful server statistics.
 - Identifies slow requests.
 - Uses thread-safe programming techniques.
 - Automatically generates a report.
 - Demonstrates practical Java programming concepts.
-

27. Limitations

- The current implementation uses a fixed number of four worker threads.
 - The slow-request threshold is fixed at 500 milliseconds.
 - The current report is generated as a text file.
 - The application processes the complete input file rather than continuously monitoring a live log.
 - No external database is used.
-

28. Future Enhancements

The project can be extended with:

- Real-time log monitoring.
- Configurable worker-thread count.
- Configurable response-time threshold.
- CSV and JSON report generation.
- Database integration.
- Graphical dashboard.
- Advanced filtering by date and log level.
- Performance comparison between single-threaded and multithreaded processing.
- Real-time alerts for critical or slow requests.

29. Learning Outcomes

After completing this project, the following concepts are demonstrated:

- Java File Handling
- Java Collections
- Regular Expressions
- Exception Handling
- Custom Exceptions
- Multithreading
- Runnable Interface
- ExecutorService
- Thread Pools
- ConcurrentHashMap
- Atomic Variables
- Synchronization
- Data Processing
- Report Generation

30. Conclusion

The **Log Analyzer – Multithreaded Log File Parser** provides an automated approach to processing and analyzing server log files.

The project demonstrates how Java multithreading can be used to divide log-processing tasks among multiple worker threads. It also demonstrates Regular Expression-based validation, custom exception handling, thread-safe data management, response-time analysis, and automated report generation.

The project combines several important Java programming concepts into a practical application for server log analysis.

31. Author

Dhairya Agrawal

Java Project – September 2026